Complaint Classifier Project — Full Summary

A plain-language recap of everything built and explained so far, so you can come back to this anytime and remember what's going on and why.

---

1. The goal

Build a small tool that reads a real customer complaint (about a bank, credit card, etc.) and automatically figures out which category it belongs to — using an LLM (Claude), wrapped in a web API, packaged with Docker. This directly targets the internship requirements: LLM experience, cloud/MLOps familiarity, and Python skills.

---

2. The data: CFPB Consumer Complaint Database

- **CFPB** = Consumer Financial Protection Bureau — a US government agency that acts like a referee between consumers and banks/lenders. It publishes a public database of real complaints people have filed.
- **What we hit:** as of **August 14, 2026**, the CFPB stopped publishing the actual written complaint text (the "narrative") in this database. Their stated reasoning: narratives are unverified one-sided claims, and publishing them was seen as low benefit / reputational risk to companies. This wasn't something you did wrong — it was a real, recent policy change.
- **The fix:** used a dataset on **Kaggle** (a free site where people share ready-made datasets) called *"Bank Customer Complaint Analysis"* — built from CFPB data *before* the narrative field was removed. It has:
  - **162,421 rows**
  - Two key columns: `narrative` (the complaint text) and `product` (the true category)
  - Five categories: `credit_reporting`, `debt_collection`, `mortgages_and_loans`, `credit_card`, `retail_banking`

What a credit report is: a record kept by companies like Equifax/Experian/TransUnion of your borrowing history — credit cards, loans, missed payments, etc. Banks check it before lending to you. Errors on it (wrong info, accounts that aren't yours) are what `credit_reporting` complaints are about. This category overlaps a lot with `debt_collection`, because a wrong report can trigger debt collectors to start contacting you — the two topics genuinely blend together in real complaint text.

---

3. Key tools and concepts explained

pandas — a Python library for working with spreadsheet-like data. Turns a CSV (plain text with commas) into a `DataFrame` — a table you can filter, sort, and query

in code.

API key — a secret password-like string tied to your Anthropic account. Every request your code sends to Claude includes this key, like a "sticky note" proving who's asking and whose credit to charge. Never share it publicly (bots scan GitHub for exposed keys).

HTTP request — the technical term for "a message sent over the internet from one computer to another." Same thing your browser does loading any website. Your script sends one to Claude's servers (`api.anthropic.com`) containing your key + your question (prompt); Claude's servers send back a response.

Environment variable (`~/.zshrc`) — instead of pasting your API key directly into code (risky if shared), it's stored in a hidden settings file on your Mac that Terminal reads automatically. Your code then reads it via `os.environ["ANTHROPIC_API_KEY"]`. This is Mac-wide, not tied to any folder — works from any Terminal window/tab, forever, once saved.

- `echo 'export ANTHROPIC_API_KEY="..."' >> ~/.zshrc` → writes the key into that settings file
- `source ~/.zshrc` → forces Terminal to re-read that file right now (normally it only reads it when a *new* window opens)

API (the one you built) — turns your script from "runs once, prints answer, exits" into "stays open, listens for requests, answers on demand." Built with **FastAPI** (defines what the service does) + **uvicorn** (actually runs it and listens for requests).

Docker — packages your code *together with* everything it needs (right Python version, right libraries) into one self-contained unit called a **container**, so it runs identically anywhere — your laptop, a cloud server, anywhere — instead of only working because your specific laptop happens to have the right setup.

---

4. The full request path (how a classification actually happens)

```
[Your browser]
     ↓ you type a complaint, hit Execute
[Your laptop — the FastAPI/uvicorn server]
     ↓ builds a prompt, sends it out using your API key
[Anthropic's servers — Claude itself]
     ↓ Claude reads the prompt, decides a category
[Anthropic's servers]
     ↓ sends the answer back
[Your laptop — FastAPI server]
     ↓ wraps it as {"category": "..."}
[Your browser]
     ↓ shows you the result
```

Your laptop acts as **both** a server (answering your browser) and a client (asking Claude) — it's the middleman. Later, deploying to the cloud just replaces "your laptop" with a cloud company's always-on server; the trip to Anthropic stays the same.

---

5. The code, explained with comments

`explore.py` — first look at the data

python

```python
import pandas as pd

# Load the CSV file into a table (DataFrame) called df
df = pd.read_csv("complaints.csv")

# Drop any row where the complaint text is empty/missing
df = df.dropna(subset=["narrative"])

# Print how many usable rows we have, and peek at the first few
print("Number of usable complaints:", len(df))
print(df[["product", "narrative"]].head())

# Count how many complaints fall into each category
print(df["product"].value_counts())
```

`classify.py` — send real complaints to Claude and check accuracy

python

```python
import os
import anthropic
import pandas as pd

# Set up your connection to Claude, using the API key saved in ~/.zshrc
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# Load the data
df = pd.read_csv("complaints.csv")

# Pick 30 random rows. random_state=42 makes it the SAME 30 rows every ru
# so results are consistent while testing (42 has no special meaning)
sample = df.sample(30, random_state=42)

correct = 0  # tally of right answers
total = 0    # tally of total attempts
```

```
# _ = the row's index number (we don't need it, so we throw it away)
# row = the actual row data (narrative + product) for this complaint
for _, row in sample.iterrows():
    narrative = row["narrative"]   # the complaint text
    true_label = row["product"]    # the correct category (answer key)

    # f"""...""" lets us drop the real narrative text into {narrative}
    prompt = f"""Classify this consumer complaint into exactly one of the
credit_reporting, debt_collection, mortgages_and_loans, credit_card, reta

Complaint: {narrative}

Respond with only the category name, nothing else."""

    # Send the prompt to Claude and get a response back
    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=20,
        messages=[{"role": "user", "content": prompt}]
    )

    # Claude's reply is nested inside response.content[0].text
    # .strip() removes any stray extra spaces
    predicted = response.content[0].text.strip()
    match = predicted == true_label

    total += 1
    if match:
        correct += 1

    print(f"True label:      {true_label}")
    print(f"Claude predicted: {predicted}")
    print(f"Match: {match}")
    print("---")

# Final score as a percentage
accuracy = correct / total * 100
print(f"\nFinal accuracy: {correct}/{total} = {accuracy:.1f}%")
```

Result: 23/30 = 76.7% (arguably ~80% — one "miss" was actually correct, Claude just added an extra explanatory sentence instead of answering with only the category name, which broke the exact-match check). Most real mistakes were `credit_reporting` vs `debt_collection` — a genuinely ambiguous pair, not a sign Claude misunderstood the task.

`api.py` — turn the script into an always-on service

python

```python
import os
import anthropic
from fastapi import FastAPI

app = FastAPI()  # creates "the shop" — the running service itself
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# This "door" (@app.get) opens at the web address /classify
# Anyone visiting it must hand over a piece of text
@app.get("/classify")
def classify(text: str):
    prompt = f"""Classify this consumer complaint into exactly one of thes
credit_reporting, debt_collection, mortgages_and_loans, credit_card, reta

Complaint: {text}

Respond with only the category name, nothing else."""

    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=20,
        messages=[{"role": "user", "content": prompt}]
    )

    predicted = response.content[0].text.strip()

    # return (not print) sends this back as the actual API response,
    # automatically converted to JSON: {"category": "credit_card"}
    return {"category": predicted}
```

Run it locally with: `uvicorn api:app --reload` Test it at: `http://127.0.0.1:8000/docs` (FastAPI's built-in interactive test page)

- `127.0.0.1` = "this exact computer" (only reachable by you, for now)
- `:8000` = the specific port/"door number" the service is listening on

`Dockerfile` — package the whole project to run anywhere

dockerfile

```
# Start from a small pre-made environment that already has Python 3.13
FROM python:3.13-slim

# Make a folder called /app inside the container and work from there
WORKDIR /app

# Copy in the list of required libraries, then install them
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy in the rest of the project (api.py, etc.)
COPY . .

# When the container starts, automatically run the API
# --host 0.0.0.0 means "accept requests from outside the container"
# (127.0.0.1 would only mean "inside the container itself")
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build it: `docker build -t complaint-classifier .` Run it: `docker run -p 8000:8000 --env ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY complaint-classifier`

Note on the version fix: the first build failed because the Dockerfile said `python:3.11-slim`, but your Mac actually runs Python 3.13 — one library (`numpy==2.5.3`) only existed for 3.13+, so it couldn't be found for 3.11. Fixed by changing the Dockerfile to `python:3.13-slim` to match.

Confirmed working: tested a real complaint ("My credit card company charged me an annual fee...") through the Dockerized API and got back `{"category": "credit_card"}` — correct, and now running fully inside a portable container instead of relying on your laptop's raw Python setup.

---

6. Progress checklist

Step	Status
Real labeled data sourced	✅ Done
LLM classification working, accuracy measured (~77-80%)	✅ Done
Wrapped in a working API (FastAPI)	✅ Done
Packaged in Docker	✅ Done
Deploy to a real cloud service (Cloud Run/App Runner/etc.)	⬜ Not started
GitHub Actions auto-deploy on push	⬜ Not started
Logging (speed, cost, failure rate)	⬜ Not started
Formal, larger evaluation test set	⬜ Optional / partially done (30-sample test exists)

Where this stands: what's built so far is already a legitimate, demoable project — real data sourcing (including navigating a live CFPB policy change), working LLM integration, a real API, and containerization. Cloud deployment is the natural next step and the one most likely to make this feel "production-grade," but everything already built stands on its own.

---

7. Cost note

Every API call to Claude in this project has cost a tiny fraction of a cent. The full 30-complaint test, plus all the manual testing through `/docs`, has used nowhere near the $5 of credit added — likely still well under $1 total. Check exact usage anytime at console.anthropic.com → Credits/Usage.